7) Turingov stroj

- **Uvod**
  - **algoritem**

    Končna množica navodil, ki vodi procesor, v končnem številu korakov, od vhodnih podatkov problema do ustrezne rešitve.
  - **računski model**

    Računski model je definicija, ki formalno opisuje osnovne pojme algoritemskega računanja (algoritem, njegovo okolje in njegovo izvajanje v okolju).

    Vsi modeli računanja so ekvivalentni v smislu: Kar je lahko izračunano z enim, je lahko tudi z drugimi.
  - **Teza o izračunljivosti (Church-Turingova teza)**

    Osnovni koncepti računanja/izračunanja so formalizirani:
    - algoritem je formaliziran z Turingovim programom
    - računanje je formalizirano z izvršitvijo Turingovega programa v Turingovem stroju
    - funkcija izračunljivosti je formalizirana z Turingovo izračunljivo funkcijo

    Teza o izračunljivosti pravi, da je vse, kar je možno izračunati v intuitivnem smislu, mogoče izračunati z enim od standardnih univerzalnih modelov (Turingov stroj, splošno rekurzivna funkcija, Postov stroj, Markovski algoritmi) računanja in obratno.
- **Turingov stroj**

  Osnovna različica Turingovega stroja ima naslednje komponente: kontrolno enoto, ki vsebuje Turingov program; trak sestavljen iz celic; in premično okno čez trak, ki je povezano z kontrolno enoto.

  TM je sedmerica $T = (Q, \Sigma, \Gamma, \delta, q_1, \mathrm{B}, F)$. Da določimo TM, moramo *določiti* $Q, \Sigma, \Gamma, \delta, F$.

  - **trak**

    Trak je uporabljen za *pisanje* in *branje* vhodnih podatkov, vmesnih podatkov, in izhodnih podatkov (rezultati). Je razdeljen v enako velike celice in je *potencialno nekončen* v eni smeri (v tej smeri je lahko *razširjen* z *končnim številom celic*).

  - **celica**

    Celica vsebuje tračni simbol, ki pripada tračni abecedi $\Gamma=\{z_1, \ldots, z_t\}, t \geq 3$.
    Simbol z_t je poseben, pomeni, da je celica *prazna*; zaradi tega razloga je označen z $\sqcup$, predsledek. Poleg $\sqcup$ obstajata še vsaj dva dodatna simbola: 0 in 1. Prevzeli bomo, da $z_1 = 0$ in $z_2 = 1$.

- **vhodni podatki**

Vhodni podatki so vsebovani v vhodni besedi. To je beseda nad vhodno abecedo Σ, tako, da je $\{0, 1\} \subseteq \Sigma \subseteq \Gamma - \{\sqcup\}$.

Vse celice so prazne (vsaka vsebuje $\sqcup$) razen *najbolj leve* celice, ki vsebujejo vhodno besedo.

- **kontrolna enota**

Kontrolna enota je vedno v nekem stanju iz končne množice stanj $Q = \{q_1, \ldots, q_s\}$, $s \geq 1$. q_1 je začetno stanje. Nekaj stanj je končnih; zbrane so v množici $F \subseteq Q$. Vsa druga stanja so *ne*-končna. Če je indeks stanja nepomemben, potem uporabljamo q_{yes} in q_{no}, ki se nanašata na končno in ne-končno stanje.

- **turingov program in funkcija prehodov**

Turingov program (TP) je v kontrolni enoti, usmerja svoje komponente.

TP je *značilen* za *določen* Turingov stroj (vsak Turingov stroj ima svoj Turingov program)

Turingov program je *parcialna/delna* funkcija δ: $\mathbf{Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}}$,

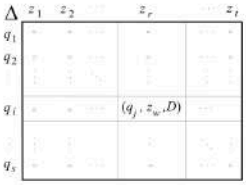
funkcija prehodov.

Turingov stroj je po definiciji determinističen, ima največ eno izbiro za premik v vsaki situaciji.

δ si lahko predstavljamo kot ***tabelo*** $\boldsymbol{\Delta = Q \times \Gamma}$, kjer

- $\Delta[q_i, z_r] = (q_j, z_w, D)$, če je $\delta(q_i, z_r) = (q_j, z_w, D)$ navodilo za δ
- $\Delta[q_i, z_r] = 0$, če je $\delta(q_i, z_r)\uparrow$(*nedoločen*)

Brez izgube na splošnosti, lahko privzamemo, da vedno obstaja prehod iz q_{no}, in noben iz q_{yes}.



- **okno**

Okno se lahko premika čez *katerokoli* celico. Kontrolna enota lahko *bere* simbol skozi okno, in *napiše* simbol skozi okno, *nadomesti* prejšnji simbol. V *enem* koraku, se lahko okno premakne do *sosednje* celice.

- **delovanje Turingovega stroja**

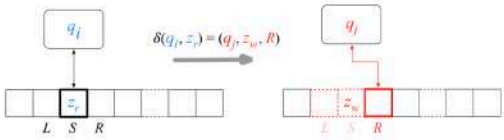
Preden se Turingov stroj začne, se mora zgoditi naslednje:

- *vhodna beseda* je zapisana *na začetku traka*
- *okno* je premaknjeno na *začetek traka*
- *kontrolna enota* je nastavljena na *začetno stanje*

Od zdaj naprej Turingov stroj deluje neodvisno, kot je ukazano z njegovim TP.

Če je TM v stanju $q_i \in Q$ in prebere simbol $z_r \in \Gamma$, potem:

- **če** je q_i končno stanje, **potem** se TM *ustavi*
- **drugače** $\delta(q_i, z_r)\uparrow$ (TP nima naslednjih navodil), **potem** se TM *ustavi*
- **drugače** $\delta(q_i, z_r)\downarrow = (q_j, z_w, D)$, **potem** TM naredi naslednje:



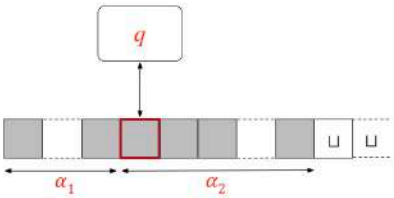
a) zamenja stanje v stanje q_j

b) zapiše z_w skozi okno

c) premakne okno na naslednjo celico v smeri $D \in \{L, R\}$ (leva ali desna), ali pusti okno, kjer je $(D = S, stay)$

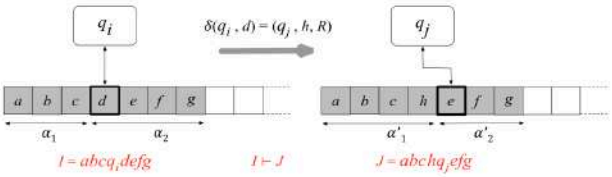
- **trenutni opis**

Trenutni opis (ID) TM je niz $I = \alpha_1 q \alpha_2$, če je trenutna konfiguracija TM takšna, da je okno nad prvim simbolom niza α_2 in da se α_2 konča na najbolj desnem ne-praznem simbolu.



ID je ``posnetek'' trenutne konfiguracije (statusa) TM komponent med zaporednimi navodili.

ID I lahko neposredno preide v J — $I \vdash J$ -- če obstaja navodilo v TM programu, čigar izvedba, I preide v J. *Refleksivno in tranzitivno zaprtje* $\vdash$ je $\vdash^*$; če je $I \vdash^* J$, potem rečemo, da ID I lahko preide v J.



- **Uporaba Turingovega stroja**
  - **računanje (vrednosti) funkcij**

Vsak TM T *inducira*, za vsak $k \geq 1$, funkcijo φ_T ki preslika k besed v 1 besedo.

Naj bo $T = (Q, \Sigma, \Gamma, \delta, q_1, \sqcup, F)$ TM in $k \geq 1$.

K-mestna lastna funkcija T - *ja* je delna funkcija $\varphi_T : (\Sigma^*)^k \rightarrow \Sigma^*$:

Če je vhod *T-ja sestavljen iz k* besed $u_1, \ldots, u_k \in \Sigma^*$, potem je vrednost φ_T pri $u_1, \ldots, u_k$ *definirana*

$\varphi_T(u_1, \ldots, u_k) :=$

- v , če se T ustavi **in** vrne na trak besedo v $\in \Sigma^*$
- $\uparrow$, *če se T* ne ustavi ***ali*** trak nima besede v Σ^*

Dana je k-mestna lastna funkcija $\varphi : (\Sigma^*)^k \rightarrow \Sigma^*$, *poišči TM T, ki lahko izračuna vrednosti* φ.

Funkcija φ (glede na to koliko močan je lahko TM)

φ je **izračunljiva** če $\exists$TM, ki lahko izračuna φ kjerkoli na dom(φ) $\wedge$ dom(φ) = $(\Sigma^*)^k$

če obstaja tak TM, ki **lahko izračuna** vrednosti φ **za vsak argument**

φ je **delno izračunljiva** če $\exists$TM, ki lahko izračuna φ kjerkoli na dom(φ)

če obstaja tak TM, ki **lahko izračuna** vrednosti φ **kadarkoli je** φ **definirana**

φ je **neizračunljiva** če *ne* obstaja TM, ki bi lahko izračunal φ kjerkoli na dom(φ)

če **ne obstaja** noben tak TM, ki bi **lahko izračunal** vrednosti φ **kadarkoli** je φ **definirana**

- **razpoznavanje množic**

Naj bo $T = (Q, \Sigma, \Gamma, \delta, q_1, \sqcup, F)$ TM in $w \in \Sigma^*$ niz.

Dana je množica $S \subseteq \Sigma^*$, poišči TM T, ki sprejme S.

Dan je jezik (množica) S, konstruirati moramo TM T, za katerega velja $L(T) = S$.

- **sprejeta beseda**

Beseda w je sprejeta z T, če $q_1 w \vdash^* \alpha_1 p\, \alpha_2$, za nek $p \in F$ in $\alpha_1\alpha_2 \in \Gamma^*$.

Če beseda povzroči, da T vstopi v končno stanje.

- **sprejet jezik**

Jezik L sprejet z T je množica $\boldsymbol{L(T) = \{w \mid w \in \Sigma^* \wedge w}$ **je sprejet z** $\boldsymbol{T\}}$.

Množica S (glede na to koliko močan je lahko TM)

S je **odločljiv**, če ∃TM, ki **odgovori z DA/NE** na "Je $x \in S$?" za **vsak** $x \in \Sigma^*$.

S je **polodločljiv,** če ∃TM, ki **odgovori z DA** na "Je $x \in S$?" **kadarkoli** $x \in S$.

S je **neodločljiv,** če **ne** obstaja **noben** TM, ki **odgovori z DA/NE** na "Je $x \in S$?" za **vsak** $x \in \Sigma^*$.

To pomeni, da *za nekaj množic S,* ***ne moremo algoritmično*** *odgovoriti* na vprašanje "Je $x \in S$?"

Zakaj?

Če je $S = L(T)$ **polodločljiv** in $x \in \Sigma^*$ poljubna beseda, potem:

- Če **je x v S**, potem se bo T sčasoma **ustavil** na vhodu x (in *sprejel* x).
- *Če* ***x ni v S****, se* T **morda ne bo ustavil** na vhodu x.

  (Ko se T izvaja, je lahko zato, ker je x eden tistih nesrečnih w-jev.)

Za tak S, dokler T *teče* na vhodu x, **ne moremo reči, ali se bo**

- T **sčasoma ustavil** (sprejme ali zavrne x), če pustimo T - ju, da teče dovolj dolgo **ali**
- bo T tekel **neskončno**.

Z drugimi besedami:

- **x ∈ S**, potem bo T *(ustavi in sprejme x)* odgovoril z **DA**
- $\boldsymbol{x \notin S}$, potem
  - T **lahko** (ustavi in zavrne x) odgovori z **NE**
  - T **lahko** (nikoli se ne ustavi) **nikoli ne odgovori z NE**

- **generiranje množic**

Dana je množica S, generiraj seznam $x_1, x_2, x_3, \ldots$ z točno elementi množice S-ja.

Dan je jezik (množica) S, konstruirati moramo TM T za katerega velja $\boldsymbol{G}(\boldsymbol{T}) = \mathbf{S}$.

Naj bo $T = (Q, \Sigma, \Gamma, \delta, q_1, \sqcup, F)$ TM. T je **generator**, če piše na trak besede iz Σ^*, v zaporedju in razmejevano (delimited) z #. (# je v Γ-Σ).

Jezik generiran z *T* je množica $\boldsymbol{G(T)} = \{\boldsymbol{w} \mid \boldsymbol{w} \in \Sigma^* \wedge \boldsymbol{T}$ **sčasoma zapiše** $\boldsymbol{w}$ **na trak**$\}$.

- **Izračunljivo preštevni jeziki (množice)**

Množica S je preštevna, če elemente S-ja lahko **naštejemo.**

Množica S je **izračunljivo preštevna** (c.e.), če je $\boldsymbol{S} = \boldsymbol{G(T)}$ za nek TM T, to je, če je S lahko generirana z Turingovim strojem.

Množica S je izračunljivo preštevna *iff S* je polodločljiva.

- **Razširitve Turingovih strojev**
  - **končni pomnilnik**

$$\delta_V : Q \times \Gamma \times \Gamma^k \to Q \times \Gamma \times \{L, R, S\} \times \Gamma^k$$

  - **večsledni trak**

$$\delta_V : Q \times \Gamma^{tk} \to Q \times \Gamma^{tk} \times \{L, R, S\}.$$

  - **dvosmerni trak**

$$\delta_V : Q \times \Gamma \to Q \times \Gamma \times \{L, R, S\}$$

  - **večtračni trak**

$$\delta_V : Q \times \Gamma^{tp} \to Q \times (\Gamma^{tk} \times \{L, R, S\})^{tp}$$

  - **večdimenzionalni trak**

$$\delta_V : Q \times \Gamma \to Q \times \Gamma \times \{L_1, R_1, L_2, R_2, \ldots, L_d, R_d, S\}.$$

- **nederministični program**

Turingov program δ dodeli vsakemu (q_i, z_r) *končno* množico *alternativnih prehodov* $\{(q_{j_1}, z_{w_1}, D_1), (q_{j_2}, z_{w_2}, D_2), \ldots\}$.

Stroj *nedeterministično* izbere prehod iz množice in ga naredi.

- **Univerzalni Turingov stroj**

  Navaden Turingov stroj, ki kot vhod sprejme opis nekega drugega Turingovega stroja in njegov vhod, ter ta stroj na podanem vhodu simulira.

- **kodiranje Turingovega stroja**

Dan je TM $T = (Q, \Sigma, \Gamma, \delta, q_1, \sqcup, F)$, želimo zakodirati T; predstaviti T z besedo nad neko kodirno abecedo.

Kako zakodiramo TM T ?

- *Kodirna abeceda* bo $\{0,1\}$.
- Zakodirali bomo samo δ, ampak tako, da bodo Q, Σ, Γ, F, ki določajo določen T, lahko obnovljeni z kodirno δ.

  Kako zakodiramo TP δ?

  1. Če je $\delta(q_i, z_j) = (q_k, z_\ell, D_m)$ navodilo za δ, potem zakodiramo navodilo z besedo $K = 0^i10^j10^k10^\ell10^m$

     kjer $D_1 = L, D_2 = R, D_3 = S$.
  2. Tako zakodiramo vsako navodilo δ.
  3. Iz obstoječih kod $K_1, K_2, \ldots, K_r$ konstruiramo kodo $\langle\delta\rangle$ δ: $\langle\delta\rangle = 111K_1 11K_2 11 \ldots 11K_r 111$

- Šifra < T > TM-ja T je zdaj identificirana z $< \delta >$ ($< T > := < \delta >$) .

- **oštevilčenje Turingovih strojev**

<T> lahko interpretiramo kot *binarno kodo* nekega *naravnega števila*. To število je indeks *T - ja*.

Naravno število katerega binarna koda ni v zahtevani obliki je indeks **praznega Turingovega stroja**.

δ tega TM-ja je *povsod nedefirana*; za vsak vhod, se ta TM *nemudoma ustavi* v 0 korakih.

Vsako naravno število je indeks točno enega Turingovega stroja.

Obnovljen TM si lahko predstavljamo kot *n-ti* osnovni TM in ga označimo z T_n .

Če gre n čez 0, 1, 2, … dobimo zaporedje Turingovih strojev.

$$T_0, T_1, T_2, \ldots$$

To je **oštevilčenje** osnovnih Turingovih strojev.

- **obstoj UTM** (in dokaz z UTM-jem -- glej ppt)

  Obstaja Turingov stroj, ki lahko izračuna vse kar je izračunljivo z kakšnim drugim Turingovim strojem.

  Turingov dokaz za obstoj Univerzalnega Turingova stroja je bil *teoretični* dokaz, da je splošno-namenski računski stroj *mogoč, vsaj v osnovi*.

  Mogoče je konstruirati *fizični* računski stroj, ki lahko računa vse kar je izračunljivo z *katerem koli drugim fizičnim računskim strojem*.

- **Izreki iz teorije izračunljivosti** (odločljivost in polodločljivost jezikov/množic)

  S *je odločljiv* $\Longrightarrow$ S *je pol-odločljiv*

  S *je odločljiv* $\Longrightarrow$ $\overline{S}$ *je odločljiv*

  S *in* $\overline{S}$ *sta polodločljiva* $\Longrightarrow$ S *je odločljiv*

  A *in* B *sta polodločljiva* $\Longrightarrow$ $A \cap B$ *in* $A \cup B$ *sta polodločljiva*

  A *in* B *sta odločljiva* $\Longrightarrow$ $A \cap B$ *in* $A \cup B$ *sta odločljiva*

8) Neodločljivost

- **Reševanje problemov**

  Naj bo D odločitveni problem.

  Naj bo d primerek problema D.

  Ponavadi smo soočeni z določenim primerkom d problema D.

  Primerek d je dobljen iz D z zamenjavo spremenljivk v definiciji D-*ja* z dejanskimi podatki.

  Problem D si lahko predstavljamo kot *množico vseh možnih primerkov D-ja*. Rečemo, da je primerek $d \in D$ *lahko* pozitiven/negativen, če je odgovor za d DA/NE.

  To je kodirna funkcija, code : $D \rightarrow \Sigma^*$, in code (D) je *množica kod vseh primerkov problema D*.

  Pišemo $\langle d \rangle$ namesto code (d).

  Naj bo code : $D \rightarrow \Sigma^*$ kodirna funkcija.

  Izberemo kode vseh pozitivnih primerkov problema D v množico $L(D)$.

  $L(D)$ je podmnožica Σ^*, se pravi je formalni jezik. Povezana je z problemom D.

- **jezik odločitvenega problema**

  Jezik odločitvenega problema D je množica $L(D)$, ki je definirana z

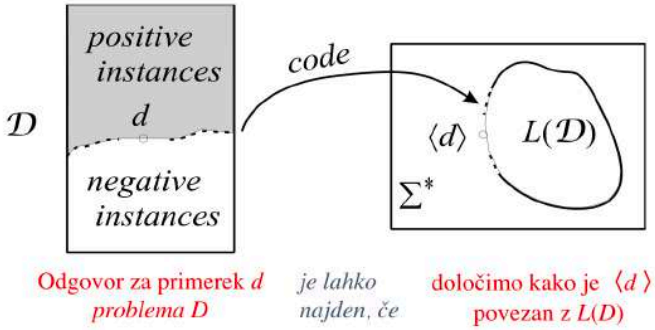
  $L(D) = \{ \langle d \rangle \in \Sigma^* \mid d$ **je pozitiven primerek problema D**$\}$.

- **povezava med odločitvenimi problemi in množicami (formalnimi jeziki)**

  $d \in D$ **je pozitiven** $\Longleftrightarrow \langle d \rangle \in L(D)$ (❊)

  Enakovrednost ❊ nam pove, da je lahko izračunavanje odgovora *primerka d* nadomestimo z *odločanjem ali je ali ni* $\langle d \rangle$ *v* $L(D)$. To je:

  Reševanje odločitvenega problema D je lahko zreducirano v razpoznavanje množice $L(D)$ v Σ^*.



- **kaj pove prepoznavnost L(D) o rešiljivosti problema D, problemi in množice**

D je **odločljiv** (***izračunljiv***) če je $L(D)$ odločljiva množica

$L(D)$ je **odločljiv** $\Rightarrow$ Obstaja algoritem, ki za *vsak* $d \in D$, odgovori z DA ali NE.

Obstaja TM, ki se za ***vsak*** **<d>** $\in \Sigma^*$, odloči, *ali je ali ni* **<d>** $\in$ ***L(D)***. Potem uporabi ❇.

D je **polodločljiv** če je $L(D)$ polodločljiva množica

$L(D)$ je **polodločljiv** $\Rightarrow$ Obstaja algoritem, ki

- za *vsako pozitivni primerek* $d \in D$, odgovori z DA
- za *negativen primerek* $d \in D$, *v končnem času* lahko odgovori z *NE* ali pa *ne odgovori z NE*

Obstaja TM, ki za **vsak** **<d>** $\in$ ***L(D)*** sprejme <d>. Če **<d>** $\notin$ ***L(D)***, algoritem *lahko zavrže ali ne zavrže* <d> *v končnem času*. Potem uporabi ❇.

D je **neodločljiv** (***neizračunljiv***) če je $L(D)$ neodločljiva množica

$L(D)$ je **neodločljiv** $\Rightarrow$ Ne obstaja algoritem, ki bi za *vsak* $d \in D$ *odgovoril z* DA ali NE.

Ne obstaja TM, ki je sposoben odločanja, za ***vsak*** **<d>** $\in \Sigma^*$, ali je ali ni **<d>** $\in L(D)$. Potem uporabi ❇.

- **Neizračunljiv problem - problem ustavitve**
  - **problem ustavljanja**

    Problem ustavljanja D_{Halt} je definiran

    D_{Halt} = **"Dan je TM T in $w \in \Sigma^*$, ali se T ustavi na w?"**

    Izrek

    Problem ustavljanja D_{Halt} je ***neodločljiv***.

    To pomeni, da *ne obstaja noben algoritem, ki bi bil* sposoben za *poljuben T in w,* odgovoriti na vprašanje "*Ali se T ustavi na/pri w?*"

    Algoritem bo spodletel za najmanj 1 par T, w.

    D_H je podproblem problema D_{Halt} (dobljen je iz D_{halt} z omejitvijo w na $w = \langle T \rangle$).

  - **univerzalni jezik**

    Univerzalni jezik K_0 je jezik *problema ustavljanja*

    $K_0 = L(D_{Halt})$ = { $\langle T, w \rangle$ / **T se ustavi na w** }

  - **diagonalni jezik**

    Diagonalni jezik K

    K = { $\langle T, T \rangle$ / **T se ustavi na** $\langle T \rangle$ }

    jezik, ki ga dobimo iz univerzalnega jezika z uvedbo omejitve w = < T >

    K je jezik problema D_H = "Dan je TM T, ali se T ustavi na $\langle T \rangle$?"

  - **dokaz problema ustavljanja** (glej ppt)

    dokaži (v lemi), da je *K neodločljiva množica*

    to bo impliciralo, da je K_0 *neodločljiv,* posledično, D_{halt} je *neodločljiv problem*

- **Osnovne vrste odločitvenih problemov**
  - **neodločljive množice, ki so polodločljive: univerzalni jezik, diagonalni jezik**

K_0 je neodločljiva (ampak še vedno) polodločljiva množica.

Najti moramo TM, ki sprejme K_0 .

Dan je poljuben vhod $\langle T, w\rangle$, stroj mora simulirati T na w, in *če* se simulacija *ustavi*, mora stroj vrniti DA in se ustaviti. Če tak stroj obstaja, bo odgovoril z DA *iff* $\langle T, w \rangle \in ?\ K_0$. Vemo, da tak stroj obstaja: to je *Univerzalni Turingov stroj U*.

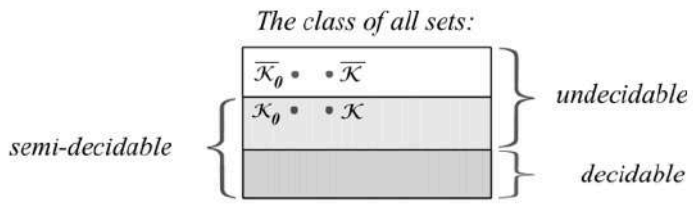
Posledično, K_0 je polodločljiva množica.

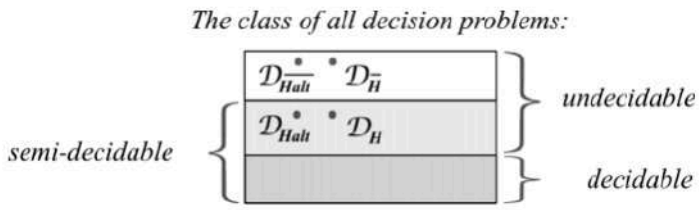
  - **neodločljive množice, ki niso polodločljive: komplement univerzalnega jezika**

$\overline{K_0}$ **ni polodločljiva množica** (komplement K_0)

Če bi bila $\overline{K_0}$ polodločljiva, potem bi bile obe K_0 in $\overline{K_0}$ polodločljive. Ampak potem bi bila K_0 odločljiva (Postov izrek). To bi bilo protislovje. Tako, da $\overline{K_0}$ *ni* polodločljiva.

  - **3 neprazni podrazredi: odločljivi, polodločljivi (vsi odločljivi, nekaj neodločljivih), neodločljive**





Množici K_0 in K sta jezika odločitvenih problemov D_{Halt} in $D_{H.}$
Množici $\overline{K_0}$, $\overline{K}$ sta jezika odločitvenih problemov:

$D_{\overline{Halt}}$ = Dan je TM T in beseda w, ali se T *nikoli ne ustavi* na w?

$D_{\overline{H}}$ = Dan je TM T, ali se T *nikoli ne ustavi na* $\langle T \rangle$?

- **3 vrste odločitvenih problemov**

Odločljiv (decidable)

To pomeni, da obstaja algoritem, ki lahko reši *poljubni primerek* $d \in D$.

Tak algoritem se imenuje **odločevalec** problema D.

Polodločljivo neodločljiv (semi-decidable undecidable)

To pomeni, da ne obstaja algoritem, ki bi rešil *poljuben primerek* $d \in D$, ampak obstaja algoritem, ki lahko reši *poljubnen pozitiven primerek* $d \in D$.

Tak algoritem se imenuje **prepoznalec** problema D.

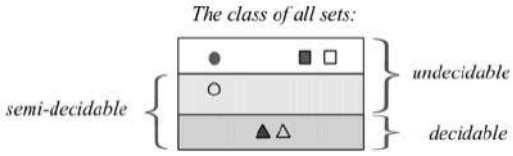
Ni polodločljiv (not semi-decidable)

To pomeni, da za *poljuben* algoritem obstaja pozitiven in negativen primerek d problema D, tak, da ju algoritem ne more rešiti.

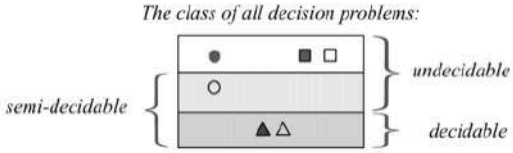
- **komplementarne množice in odločitveni problemi**

Obstajajo samo 3 možnosti za **odločljivost množice S in njegovega komplementa** $\overline{S}$:

- S in $\overline{S}$ sta ***odločljiva*** (△ ▲)
- S in $\overline{S}$ sta ***neodločljiva***; *en je* ***polodločljiv*** in *drugi ni* (○●)
- S in $\overline{S}$ sta ***neodločljiva*** in **noben** *od njiju ni polodločljiv* (□ ■)



Enako velja za **odločljivost odločitvenega problema**:



Glede na izid procesiranja lahko **polodločljive jezike** nad dano abecedo razdelimo bolj natančno.

Poseben podrazred tvorijo **odločljivi jeziki**: to so jeziki, za katere obstaja tak Turingov stroj, ki ne glede na vhodni niz vedno konča procesiranje v končnem stanju ($q_{sprejmi}$), če je vhodni niz vsebovan v jeziku TM-ja, bodisi v stanju (q_{zavrni}), če vhodni niz ni vsebovan.

Pri **polodločljivih jezikih, ki niso odločljivi**, gre za jezike, ki jih prepozna neki TM, vendar se pri vhodnem nizu, ki ne pripada jeziku TM-ja, procesiranje konča bodisi v stanju (q_{zavrni}) bodisi se ne ustavi = ni mogoče odločiti, ali vhodna beseda pripada jeziku ali ne: če se stroj ne ustavi, ne vemo, ali se ustavi, ker niza ni v jeziku TM-ja, ali pa se ne ustavi, ker ni dovolj dolgo procesiral.

Ali je razred polodločljivih jezikov kar enak razredu odločljivih jezikov? Z drugimi besedami, ali lahko za vsak jezik, ki ga razpozna neki TM, oblikujemo tak TM, ki se bo vedno ustavil in ki bo prepoznal prav ta jezik? Če je to res, bi to pomenilo, da je vsak jezik polodločljiv. A to ni res. Obstajajo polodločljivi, vendar neodločljivi jeziki.

- **Prevedbe**
  - diagonalni jezik je neodločljiv
  - diagonalni jezik ni polodločljiv
  - komplement diagonalnega jezika je polodločljiv, vendar neodločljiv
  - univerzalni jezik je neodločljiv in polodločljiv
    (neodločljivost jezika L_u dokažemo s prevedbo jezika komplement L_d na L_u)
  - neprazni jezik je neodločljiv (L_u na L_{ne})
  - neprazni jezik je polodločljiv
  - jezik ustavljanja je neodločljiv

  - odločljivi jeziki so zaprti nad komplementom
    Če je jezik L odločljiv, je njegov komplement tudi odločljiv.
    Jezik L je odločljiv natanko takrat, ko sta L in njegov komplement polodločljiva.

- **Problem garača**

Definicija

$\mathcal{T}_n$ (za $n \geq 1$) je razred vseh TM, ki imajo:

- neomejen trak v obe smeri
- n ne-končnih stanj (vključno q_1) in 1 končno stanje q_{n+1}
- $\Sigma = \{0,1\}$ in $\Gamma = \{0,1,\sqcup\}$
- δ piše samo simbol 1 in premika okno v levo ali v desno

Izrek (Radó)

Za vsak $n \geq 1$, obstaja *končno mnogo* TM v $\mathcal{T}_n$.

TM $T \in \mathcal{T}_n$ je stroj, ki se ustavi, če se *T ustavi* pri *praznem vhodu*.

Izrek (Radó)

Za *vsak* $n \geq 1$, *obstaja* stroj, ki se ustavi v $\mathcal{T}_n$.

Posledično obstaja *najmaj 1* in največ *končno mnogo* ($|\mathcal{T}_n|$) strojev, ki se ustavijo v $\mathcal{T}_n$.

V $\mathcal{T}_n$ mora obstajati stroj, ki se ustavi in ima, med vsemu stroji, ki se ustavijo v $\mathcal{T}_n$ *maksimalno število enic*, ki ostanejo na traku *po ustavitvi*.

Tak stroj, ki se ustavi, se imenuje *n*-stanjski *garač* in ga označimo z *n*-BB.

Problem barača

Naj bo $T \in \bigcup_{i \geq 1} \mathcal{T}_i$ poljuben TM.

"Ali je T garač?" (``Ali obstaja n ≥ 1, za katerega velja T = n-BB?")

Funkcija barača je $s(n)$ = 'število enic pridobljenih z n-BB'

Izrek

Funkcija barača je ***neizračunljiva***.

10) Teorija računske zahtevnosti

- **Deterministična časovna zahtevnost (DTIME)**

Naj bo $M = (Q, \Sigma, \Gamma, \delta, q_1, \sqcup, F)$ DTM z $k \geq 1$ dvosmerimi neskončnimi trakovi.

- **deterministična časovna zahtevnost**

M je determinističnе časovne zahtevnosti T(n), če za vsak vhod $w \in \Sigma^*$ dolžine n,

M naredi $\leq T(n)$ korakov pred ustavitvijo.

Privzeto je, da M prebere vse w - je … $T(|w|) \geq |w| + 1$, $T(n) \geq n + 1$.

Če je T(n) najmanj linearen.

M časovne zahtevnosti T(n) se lahko odloči $w \in ?$ L(M) v $\leq T(|w|)$ korakih.

- **jezik determinističnе časovne zahtevnosti**

Jezik L je determinističnе časovne zahtevnosti T(n), če obstaja DTM M determinističnе časovne zahtevnosti T(n), da velja L = L(M).

- **razred vseh jezikov determinističnе časovne zahtevnosti**

DTIME(T(n)) = {L | L je jezik ∧ L je determinističnе časovne zahtevnosti T (n)}

Vsebuje vse L-je za katere je problem $w \in ?$ L lahko deterministično rešljiv v $\leq T(|w|)$ času.

- **odločitveni problem determinističnе časovne zahtevnosti**

Odločitveni problem D je determinističnе časovne zahtevnosti T(n), če ima njegov jezik L(D) deterministično časovno zahtevnost T(n).

- **razred vseh odločitvenih problemov determinističnе časovne zahtevnosti**

DTIME(T(n)) = {D | D je odločitveni problem ∧ D je determinističnе časovne zahtevnosti T (n)}

Vsebuje vse D - je čigar primerki d so lahko deterministično rešljivi v $\leq T(|\langle d \rangle|)$ času.

- **Deterministična prostorska zahtevnost (DSPACE)**

Naj bo $M = (Q, \Sigma, \Gamma, \delta, q_1, \sqcup, F)$ DTM z 1 vhodnim trakom in $k \geq 1$ delovnimi trakovi.

- **deterministična prostorska zahtevnost**

M je deterministične prostorske zahtevnosti S(n), če, za vsak vhod $w \in \Sigma^*$ dolžine n, M pred zaustavitvijo uporabi $\leq S(n)$ celic na vsakem delovnem traku.

- Celice vhodnega traka niso pomembne.
- Predpostavljeno je, da M uporabi najmanj celico pod začetnim položajem okna. S(n) je najmanj konstantna funkcija 1.

M deterministične prostorske zahtevnosti S(n) se lahko odloči $w \in ?\ L(M)$ na prostoru $\leq S(n)$.

- **jezik deterministične prostorske zahtevnosti**

Jezik L je deterministične prostorske zahtevnosti S(n), če obstaja DTM M deterministične prostorske zahtevnosti S(n), da velja L = L(M).

- **razred vseh jezikov deterministične prostorske zahtevnosti**

DSPACE(S(n)) = {L | L je jezik ∧ L je deterministične prostorske zahtevnosti S (n)}

Vsebuje vse L-je za katere je lahko problem $w \in ?\ L$ deterministično rešljiv na $\leq S(|w|)$ prostoru.

- **odločitveni problem deterministične prostorske zahtevnosti**

Odločitveni problem D je deterministične prostorske zahtevnosti S(n), če ima njegov jezik L(D) deterministično prostorsko kompleksnost S(n).

- **razred vseh odločitvenih problemov deterministične prostorske zahtevnosti**

DSPACE(S(n)) = {D | D je odločitveni problem ∧ D je deterministične prostorske zahtevnosti S(n)}

Vsebuje vse D-je čigar primerki d so lahko deterministično rešljivi na $\leq S(|\langle d \rangle|)$ prostoru.

- **Nedeterministična časovna zahtevnost (NTIME)**

Naj bo $N = (Q, \Sigma, \Gamma, \delta, q_1, \sqcup, F)$ nedeterministični NTM.

- **nedeterministična časovna zahtevnost**

N je nedeterministične časovne zahtevnosti T(n), če za vsak vhod $w \in \Sigma^*$ dolžine n, obstaja izračun v katerem N naredi $\leq T(n)$ korakov pred ustavitvijo.

Predpostavljeno je, da N prebere vse w-je … $T(|w|) \geq |w| + 1$, $T(n) \geq n + 1$.

T(n) je najmanj linearna funkcija.

N časovne zahtevnosti T(n) se lahko odloči $w \in ?\ L(M)$ v $\leq T(|w|)$ korakih.

- **jezik nedeterministične časovne zahtevnosti**

Jezik L je nedeterministične časovne zahtevnosti T(n), če obstaja nedeterministični NTM N nedeterministične časovne zahtevnosti T(n) tako, da velja L = L(M).

- **razred vseh jezikov nedeterministične časovne zahtevnosti**

NTIME(T (n)) = {L | L je jezik ∧ L je nedeterministične časovne zahtevnosti T (n)}

Vsebuje vse L - je za katere je lahko problem $w \in ?\ L$ nedeterministično rešljiv v $\leq T(|w|)$ času.

- **odločitveni problem nedeterministične časovne zahtevnosti**

Odločitveni problem D je nedeterministične časovne zahtevnosti T(n), če je njegov jezik L(D) nedeterministične časovne zahtevnosti T(n).

- **razred vseh odločitvenih problemov nedeterministične časovne zahtevnosti**

NTIME(T(n)) = {D | D je odločitveni problem ∧ D je nedeterministične časovne zahtevnosti T (n)}

Vsebuje D - je čigar primerki d so lahko nedeterministično rešljivi v $\leq T(|\langle d \rangle|)$ času.

- **Nedeterministična prostorska zahtevnost (NSPACE)**

Naj bo $N = (Q, \Sigma, \Gamma, \delta, q_1, \sqcup, F)$ nedeterministični NTM z 1 vhodnim trakom in $k \geq 1$ delovnimi trakovi.

- **nedeterministična prostorska zahtevnost**

N je nedeterministične prostorske zahtevnosti S(n), če za vsak vhod $w \in \Sigma^*$ dolžine n, obstaja izračun v katerem N pred ustavitvijo uporabi $\leq S(n)$ celic na vsakem delovnem traku.

- Celice vhodnega traka niso pomembne.
- Predpostavljeno je, da M uporabi najmanj celico pod začetno pozicijo okna. S(n) je najmanj konstanta funkcija 1.

N z prostorsko zahtevnostjo S(n) se lahko odloči $w \in ?\ L(M)$ na $\leq S(|w|)$ prostoru.

- **jezik nedeterministične prostorske zahtevnosti**

Jezik L je nedeterministične prostorske zahtevnosti S(n), če obstaja nedeterminističen NTM N nedeterministične prostorske zahtevnosti S(n) tako, da velja L = L(M).

- **razred vseh jezikov nedeterministične prostorske zahtevnosti**

NSPACE(S (n)) = {L | L je jezik $\wedge$ L je nedeterministične prostorske zahtevnosti S (n)}

Vsebuje vse L - je za katere $w \in ?$ L je lahko nedeterministično rešljiv na $\leq S(|w|)$ prostoru.

- **odločitveni problem nedeterministične prostorske zahtevnosti**

Odločitveni problem D je nedeterministične prostorske zahtevnosti S(n), če je njegov jezik L(D) nedeterministične prostorske zahtevnosti S(n).

- **razred vseh odločitvenih problemov nedeterministične prostorske zahtevnosti**

NSPACE(S(n)) = {D | D je odločitveni problem $\wedge$ D je nedeterministične prostorske zahtevnosti S (n)}

Vsebuje vse D-je čigar primerki d so lahko nedeterministično rešljivi na $\leq S(|\langle d \rangle|)$ prostoru.

- **Povzetek razredov zahtevnosti**

$\mathbf{DTIME(T(n))} = \{L \mid L \text{ je jezik} \wedge L \text{ je časovne zahtevnosti } T(n)\}$

$\mathbf{DSPACE(S(n))} = \{L \mid L \text{ je jezik} \wedge L \text{ je prostorske zahtevnosti } S(n)\}$

$\mathbf{NTIME(T(n))} = \{L \mid L \text{ je jezik} \wedge L \text{ je nedeterministične časovne zahtevnosti } T(n)\}$

$\mathbf{NSPACE(S(n))} = \{L \mid L \text{ je jezik} \wedge L \text{ je nedeterministične prostorske zahtevnosti } S(n)\}$

$\mathbf{DTIME(T(n))} = \{D \mid D \text{ je odločitveni problem} \wedge L(D) \text{ je časovne zahtevnosti } T(n)\}$

$\mathbf{DSPACE(S(n))} = \{D \mid D \text{ je odločitveni problem} \wedge L(D) \text{ je prostorske zahtevnosti } S(n)\}$

$\mathbf{NTIME(T(n))} = \{D \mid D \text{ je odločitveni problem} \wedge L(D) \text{ je nedeterministične časovne zahtevnosti } T(n)\}$

$\mathbf{NSPACE(S(n))} = \{D \mid D \text{ je odločitveni problem} \wedge L(D) \text{ nedeterministične prostorske zahtevnosti } S(n)\}$

$\mathbf{DTIME(T(n))} = \{\text{odločitveni problemi deterministično rešljivi v času } T(n)\}$

$\mathbf{DSPACE(S(n))} = \{\text{odločitveni problemi deterministično rešljivi na prostoru } S(n)\}$

$\mathbf{NTIME(T(n))} = \{\text{odločitveni problemi nedeterministično rešljivi v času } T(n)\}$

$\mathbf{NSPACE(S(n))} = \{\text{odločitveni problemi nedeterministično rešljivi na prostoru } S(n)\}$

- **Stiskanje trakov**

  Prostorska zahtevnost lahko vedno zmanjšana z konstantnim faktorjem (z zakodiranjem večih tračnih simbolov v enega). Z razširitvijo abeced, zmanjšamo prostor.

  Izrek

  Če ima L prostorsko zahtevnost $S(n)$, potem za vsak $c > 0$, *ima* L časovno zahtevnost $c\,S(n)$. To drži tudi za nedeterministično prostorsko zahtevnost.

  Posledica:

  Za *vsak* $c > 0$ je **DSPACE($S(n)$) = DSPACE($cS(n)$)** in **NSPACE($S(n)$) = NSPACE($cS(n)$)**.

- **Linearna pohitritev**

  Časovno zahtevnost lahko vedno zmanjšana z konstantim faktorjem (z združitvijo nekaj korakov v enega).

  Izrek

  Naj bo $\inf_{n\to\infty} T(n)/n = \infty$ in $k > 1$. Potem:

  Če ima L časovno zahtevnosti $T(n)$, potem ima za vsak $c > 0$, L časovno zahtevnost $cT(n)$. To drži tudi za nedeterministično prostorsko zahtevnost.

  Posledica

  Če je $\inf T(n)/n = \infty$, potem za *vsak* $c > 0$ velja:

  DTIME($T(n)$) = DTIME($cT(n)$) in **NTIME($T(n)$) = NTIME($cT(n)$)**.

Pod določenimi pogoji:

$\mathrm{DTIME}(T(n)) = \mathrm{DTIME}(cT(n))$

$\mathrm{NTIME}(T(n)) = \mathrm{NTIME}(cT(n))$

$\mathrm{DSPACE}(S(n)) = \mathrm{DSPACE}(cS(n))$

$\mathrm{NSPACE}(S(n)) = \mathrm{NSPACE}(cS(n))$

Pozitivna konstanta c *nima nobenega vpliva na* vsebino razreda.

Primer: $\mathrm{DTIME}(n^2) = \mathrm{DTIME}(0.33\, n^2) = \mathrm{DTIME}(4n^2) = \mathrm{DTIME}(7n^2) = \ldots$

Namesto pisanja, da je odločitveni problem D v $\mathrm{DTIME}(n^2)$, lahko rečemo, da:

D ima deterministično časovno zahtevnost of the order $O(n^2)$.

- **Zmajševanje števila trakov**

Da ugotovimo *časovno zahtevnost* uporabimo TM z $k \geq 1$ trakovi.
Vprašanje: Kako *zmanjševanje* števila k vpliva na *časovno zahtevnost*?
Odgovor: Če omejimo TM na 1 trak, lahko časovna zahtevnost *postane kvadratna*, ampak če jih omejimo na 2 traka, bo izguba časa manjša.

Izrek

Če je $L \in \mathrm{DTIME}(T(n))$, potem je L sprejet v času $O(T^2(n))$ z 1-tračnim TM.
Če je $L \in \mathrm{NTIME}(T(n))$, potem je L sprejet v času $O(T^2(n))$ z 1-tračnim NTM.
Če je $L \in \mathrm{DTIME}(T(n))$, potem je L sprejet v času $O(T(n) \log T(n))$ z 2-tračnima TM.
Če je $L \in \mathrm{NTIME}(T(n))$, potem je L sprejet v času $O(T(n) \log T(n))$ z 2-tračnima NTM.

Da ugotovimo *prostorsko zahtevnost* uporabimo TM z $k \geq 1$ delovnimi trakovi in 1 vhodnim trakom.

Vprašanje: Kako *zmanjševanje* števila k vpliva na *prostorsko zahtevnost?*
Answer: Zmanjševanje števila trakov ne vpliva na prostorsko zahtevnost.

Izrek

Če je L sprejet z k-delovnimi traki TM prostorske zahtevnosti $S(n)$, potem je L sprejet z 1-delovnim trakom TM prostorske zahtevnosti $S(n)$.

- **Relacija med DTIME, DSPACE, NTIME, NSPACE**
  - **relacije med različnimi razredi zahtevnosti**

    DTIME, DSPACE

    $DTIME(T(n)) \subseteq DSPACE(T(n))$

    Kar je lahko rešljivo v *času* $O(T(n))$, je lahko rešljivo tudi na *prostoru* $O(T(n))$.

    DSPACE, DTIME

    $L \in DSPACE(S(n)) \wedge S(n) \geq \log_2 n \Rightarrow \exists c : L \in DTIME(c^{S(n)})$

    Kar je lahko rešljivo na *prostoru* $O(S(n))$, je lahko tudi rešljivo *(največ) v času* $O(c^{S(n)})$. (Tukaj je c odvisen od L.)

    NTIME, DTIME

    $L \in NTIME(T(n)) \Rightarrow \exists c : L \in DTIME(c^{T(n)})$

    Kar je lahko rešljivo v *nedeterminističnem času* $O(T(n))$, je lahko rešljivo v *največ determinističnem času* $O(c^{T(n)})$. Posledično, zamenjava nedeterminističnega algoritma z determinističnim povzroči največ eksponentno rast v času, ki je potreben za reševanje problema.

    NSPACE, DSPACE

    $NSPACE(S(n)) \subseteq DSPACE(S^2(n))$, *if* $S(n) \geq \log_2 n \wedge S(n)$ is ``well-behaved."

    Kar je lahko rešljivo na *nedeterminističnem prostoru* $O(S(n))$, je lahko rešljivo na *determinističnem prostoru* $O(S^2(n))$. Posledično, zamenjava nedeterminističnega algoritma z determinističnim povzroči največ kvadratno rast na prostoru, ki je potreben za reševanje problema.

  - **(popolnoma) prostorska predstavljiva funkcija**

    Funkcija $S(n)$ je prostorsko predstavljiva, če obstaja TM M prostorske zahtevnosti $S(n)$, da za vsak n, *obstaja vhod dolžine n* na katerem M uporabi točno $S(n)$ tračnih celic. Če za vsak n, M uporabi točno $S(n)$ celic za *vsak vhod dolžine n*, potem rečemo, da je $S(n)$ popolnoma prostorsko predstavljiva.

  - **(popolnoma) časovna predstavljiva funkcija**

    Funkcija $T(n)$ je časovno predstavljiva, če obstaja TM M časovne zahtevnosti $T(n)$, da za vsak n, *obstaja vhod dolžine n* na katerem M uporabi točno $T(n)$ prehodov. Če *za vse n*, M naredi točno $T(n)$ prehodov za *vsak vhod dolžine n*, potem rečemo, da je $T(n)$ popolnoma časovno predstavljiva.

- **P, NP, PSPACE, NPSPACE**

$P = \cup_{i \geq 1} DTIME(n^i)$

je razred vseh odločitvenih problemov rešljivih *v determinističnem polinomskem času*

$NP = \cup_{i \geq 1} NTIME(n^i)$

je razred vseh odločitvenih problemov rešljivih *nedeterministično v polinomskem času*

$PSPACE = \cup_{i \geq 1} DSPACE(n^i)$

je razred vseh odločitvenih problemov *determinističmo* rešljivih na *polinomskem prostoru*

$NPSPACE = \cup_{i \geq 1} NSPACE(n^i)$

je razred vseh odločitvenih problemov *nedeterministično* rešljivih na *polinomskem prostoru*

i (1 do neskončnosti, velikost primerka, ki se rešuje; npr. n^2)

Izrek: $P \subseteq NP \subseteq PSPACE = NPSPACE$

Dokaz

- **($P \subseteq NP$)**

  Vsak deterministični TM polinomske časovne zahtevnosti si lahko predstavljamo kot (trivialni) *ne*deterministični TM enake časovne zahtevnosti.

- **($NP \subseteq PSPACE$)**

  Če je $L \in NP$, potem obstaja tak $\exists k$, *da je* $L \in NTIME(n^k)$.

  Zaradi izreka je $L \in NSPACE(n^k)$, in posledično (Savitch) je $L \in DSPACE(n^{2k})$. Potem je $L \in PSPACE$.

- **(PSPACE = NPSPACE)**

  $PSPACE \subseteq NPSPACE$

  Druga smer: NSPACE = (def) = $\cup$ $NSPACE(n^i) \subseteq$ (Savitch) $\cup$ $DSPACE(n^i) \subseteq PSPACE$.

Dokazali smo: PSPACE = NPSPACE.

Če je prostorska zahtevnost polinomska, nedeterminizem ne doda nič k računski moči.

Ali podobno velja tudi za *čas*? $P \subseteq NP$

Ali je: $P = NP$?

Ali je: $P \subsetneq NP$?

- **Vprašanje P =? NP**
  - **pomembnost vprašanja**

Veliko *pomembnih odločitvenih problemov* je v NP. Vsak tak problem D ima *nedeterministični algoritem* N_D, ki reši D v *nedeterminističnem polinomskem času*.

Ampak nedeterministični algoritmi veljajo za *nerealistične*, ker jih ne more izvesti noben realen računalnik. (Kako bi lahko realni računalnik *vedno izbral pravo odločitev iz nekaj možnih izbir?)*

N_D moramo zamenjati z *ekvivalentnim determinističnim algoritmom* A_D, ki izračuna enak rezultat kot N_D z *simulacijo vsake nedeterministične izbire* N_D z zaporedjem *determinističnih korakov*.

A_D zahteva *dodaten čas* (v primerjavi z N_D), da pridobi enak rezultat.
Ampak, koliko časa A_D potrebuje, da reši D?

$L \in \text{NTIME}(T(n)) \Rightarrow \exists c : L \in \text{DTIME}(c^{T(n)})$. Izrek nam pove, da zamenjava nedeterminističnega algoritma z determinističnim povzroči *največ eksponentno* rast potrebnega časa za rešitev problema.

V našem primeru, $D \in \text{NP}$, $D \in \text{NTIME}(n^k)$ za nek $k \geq 1$. $D \in \text{DTIME}(c^{n^k})$. Posledično, D je deterministično rešljiv v času c^{n^k}. Z drugimi besedami, A_D potrebuje <u>*največ*</u> $O(c^{n^k})$ časa, da reši D.

Ali lahko A_D, *kljub zgornji meji* $O(c^{n^k})$, reši D v *determinističnem polinomskem času*?

Če lahko, ali je to res za *poljuben* $D \in \text{NP}$? Vprašanje je ekvivalentno "Is P=NP?"

Če je P = NP, potem je *vsak* $D \in \text{NP}$ *deterministično* rešljiv v polinomskem času.
Vprašanje "Is P = NP?" je lahko zastavljeno tudi takole:

Je res, da če je časovna zahtevnost polinomska, potem nedeterminizem ne doda nič k računski moči?

- **dokaz P $\subsetneq$ NP, prevedba problema**

Kako? Pomembna metoda:

1. najdi *najtežji* problem v NP
2. dokaži, da ta problem *ni* v P

Metoda temelji na naši *intuiciji* kar namiguje na:

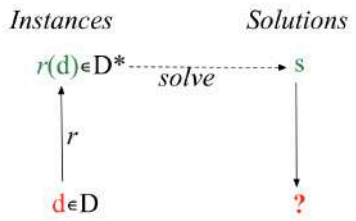
- če obstaja *kakšen* problem v NP−P, potem bo *najtežji* problem v NP med njimi
- lažje je dokazati, da tega *najtežjega* problema v NP ni v P, potem pa dokazati, da nekega drugega (*manj težjega*) problema v NP ni v P

Kdaj je *najtežji* problem v NP? Kako to definiramo?

D^* je *najtežji problem v* NP, *če vsak* $D \in$ NP *je največ tako težek kot* D^*.

Ideja: *Predpostavimo*, da *obstaja* $D^* \in$ NP, tako, da lahko z "lahkoto" zmanjšamo vsak $D \in$ NP v ta D^* v naslednjem smislu:

- mora obstajati funkcija $r : D \rightarrow D^*$
- ki lahko "z lahkoto" preoblikuje katerikoli primerek $\boldsymbol{d} \in \boldsymbol{D}$ v primerek $\boldsymbol{r(d)} \in D^*$
- tako, da je rešitev $\boldsymbol{s}$ to $\boldsymbol{r(d)}$ z "lahkoto" preoblikovana v rešitev "**?**" to $\boldsymbol{d}$



Za vsak problem D, je reševanje D-*ja* lahko ``z lahkoto'' *zamenjano* z reševanjem D^* - ja.

Če bi bilo to možno, potem bi lahko na vsak D gledali kot *največ tako težek kot* D^*.

Z drugimi besedami, na D^* lahko gledamo kot na *najtežji* problem v NP.

"Z lahkoto (easily)'" == "*v determinističnem polinomskem času*'"

- **polinomsko-časovno prevedljiv problem, polinomsko-časovna prevedba**

(zmanjševanje polinomskega časa)

Problem $D \in$ NP je polinomsko-časovno prevedljiv na problem D', $D \leq^p D'$, če obstaja deterministični TM M polinomske časovne zahtevnosti, ki za vsak $d \in D$, vrne $d' \in D'$, tako, da je, *d pozitiven* $\Longleftrightarrow$ *d' je pozitiven*. Relacija $\leq^p$ je polinomsko-časovna prevedba.

M zamenja, v polinomskem času, $d \in D$ z $d' \in D'$ ki ima enak odgovor kot d.
(M vzame $<d>$ in v polinomskem času vrne besedo $M(<d>)$, kjer je
$<d> \in L(D) \Longleftrightarrow M(<d>) \in L(D')$.

- **NP - težek problem D***

  če $D \leq^p D^*$, za vsak $D \in$ NP.

- **NP - poln problem D***

  ***Najtežji* problem v NP (*NP - poln problem*)** je lahko katerikoli problem D* z značilnostmi:

  - $D^* \in$ NP
  - $D \leq^p D^*$, za vsak $D \in$ NP

  Posledično, D^* je NP-poln, *če* je D^* v NP *in* D^* je NP-težek. -

  NP-*težek* D^* *je ali ni v* NP.

- **NP-poln problem**

  SAT (problem izpolnjivosti)

- **dokazovanje NP-polnih problemov**

  Izrek

  Naj bo $D \leq^p D'$. Potem:

  - $D' \in \mathrm{P} \Rightarrow D \in \mathrm{P}$
  - $D' \in \mathrm{NP} \Rightarrow D \in \mathrm{NP}$.

  Vsak problem *D, ki je lahko* $\leq^p$-zmanjšan na problem v P (ali v NP), je tudi v P (ali v NP).

  Izrek

  Relacija $\leq^p$ je tranzitivna.

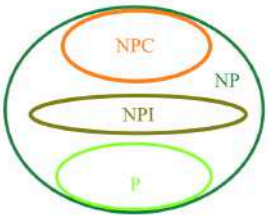
  Z drugimi besedami: $D \leq^p D' \wedge D' \leq^p D'' \Rightarrow D \leq^p D''$.

  Posledica

  - D^* je NP-težek $\wedge$ $D^* \leq^p D^{☆} \Rightarrow D^{☆}$ je NP-težek
  - D^* je NP-poln $\wedge$ $D^* \leq^p D^{☆} \wedge D^{☆} \in$ NP $\Rightarrow D^{☆}$ je NP-poln

- **povzetek**

Če je P ≠ NP, *potem* je situacija v razredu NP opisana spodaj:



If P ≠ NP

NPC je razred vseh NP-*polnih* problemov.

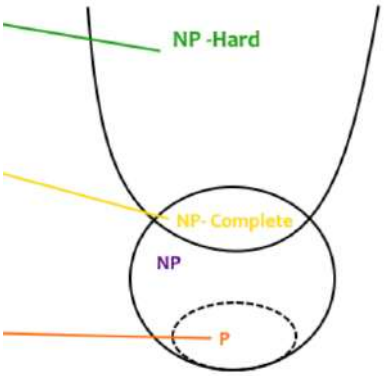
NPI je razred vseh NP-*vmesnih* problemov.

Če je P≠NP, potem obstaja problem v NP, ki ni niti v P niti v NPC.

Tak problem se imenuje NP-vmesni problem.

Primer: Ali je dano naravno število sestavljeno?

Če je P≠NP, *potem* noben problem v NPC ali NPI nima zahtevnosti polinomskega časa.



Here P != NP